Architettura e Generazione delle Stanze

Analisi della struttura dati (es. BallRoomEast.asm), codifica Layout a 2-bit e mapping delle Tiles

In **Jet Set Willy** per ZX Spectrum, la gestione della memoria è un elemento fondamentale del design. Ogni stanza del gioco occupa una struttura dati binaria altamente ottimizzata di esattamente **286 byte**. File d'ambiente come BallRoomEast.asm definiscono l'intera geografia, la grafica delle tile, gli attributi colore, gli elementi interattivi (rampe, nastri trasportatori, oggetti raccoglibili) e i nemici (guardiani).

1. Anatomia della Struttura Dati della Stanza (286 Byte)

La struttura dati di ogni stanza (ad esempio BallRoomEastLayout) è suddivisa in sezioni fisse. Di seguito è riportata la mappa completa della memoria della stanza:

Offset (Byte)	Dimensione	Sezione Dati (Simbolo)	Descrizione e Ruolo nel Gioco
`0 - 127`	128 Byte	`Layout`	Griglia 32x16 compressa (2 bit per cella). Definisce i 4 tipi di tile base.
`128 - 159`	32 Byte	`Tile (Name)`	Stringa ASCII di 32 caratteri con il nome della stanza mostrato in basso.
`160 - 213`	54 Byte	`Tiles Graphic`	6 tipi di Tile (9B ciascuno): Background, Floor, Wall, Nasty, Ramp, Conveyor.
`214 - 217`	4 Byte	`ConveyorData`	Parametri nastro trasportatore: Direzione (1B), Location VRAM (2B), Lunghezza (1B).
`218 - 221`	4 Byte	`RampData`	Parametri rampa: Direzione pendenza (1B), Location VRAM (2B), Lunghezza (1B).
`222`	1 Byte	`BorderColor`	Colore del bordo dello schermo ZX Spectrum per questa stanza (0..7).
`223 - 226`	4 Byte	`Reserved/Colors`	Palette colori per i nemici 1-4.
`227 - 234`	8 Byte	`Item Graphic`	Bitmap 8x8 dell'oggetto raccoglibile presente nella stanza.
`235 - 238`	4 Byte	`Direction`	ID delle stanze adiacenti per i cambi stanza (Sinistra, Destra, Sopra, Sotto).
`239`	1 Byte	`Physics Byte`	Configurazione fisica specifica (%GGGGJJJJ: Gravità e Forza Salto).
`240 - 279`	40 Byte	`Enemies 1-8`	8 definizioni nemici (5B ciascuno): Tipo/Flag, Sprite, Pos Min, Pos Max, Speed.
`280 - 281`	2 Byte	`PosWilly`	Coordinata iniziale di Willy all'ingresso della stanza (X, Y in pixel).
`282 - 285`	4 Byte	`EnemyColors5_8`	Palette colori per i nemici 5-8.

2. La Griglia di Layout e la Compressione a 2-Bit

La schermata di gioco dello ZX Spectrum per ogni stanza ha una griglia di **32 colonne x 16 righe = 512 celle** (tiles). Per rappresentare 512 celle senza consumare 512 byte di RAM, Jet Set Willy utilizza un sistema di compressione a **2 bit per cella**:

- **512 celle × 2 bit = 1024 bit = 128 byte**.

Ogni byte della sezione Layout contiene esattamente **4 celle adiacenti** ordinate da sinistra a destra.

Valore 2-Bit	ID Decimale	Tipo Tile	Comportamento Fisico nel Gioco
`00`	0	**Background**	Spazio vuoto. Willy e i nemici possono attraversarlo liberamente.
`01`	1	**Floor**	Piattaforma/Pavimento solido. Supporta i piedi di Willy (camminata).
`10`	2	**Wall**	Muro solido. Blocca il movimento sia orizzontale che verticale.
`11`	3	**Nasty**	Elemento mortale (spine, candele, fiamme). Causa la morte di Willy al tocco.

Esempio Pratico da BallRoomEast.asm:
Esaminando la Riga 5 di BallRoomEastLayout:
DEFB 85, 85, 90, 10, 170, 170, 170, 170
Prendiamo il valore decimale 90 (%01011010 in binario):
- Bit 7-6 = 01 (Floor) → Prima cella
- Bit 5-4 = 01 (Floor) → Seconda cella
- Bit 3-2 = 10 (Wall) → Terza cella
- Bit 1-0 = 10 (Wall) → Quarta cella

Questo singolo byte imposta 2 tile Floor seguiti da 2 tile Wall!

3. Relazione tra la Definizione delle Tiles e il Layout

Il Layout memorizza soltanto l'**indice logico** (0, 1, 2 o 3) per ogni cella della griglia. Per poter disegnare la riga a schermo con la grafica ed i colori corretti, il motore di gioco mette in relazione ciascun indice logico con il relativo blocco grafico definito a partire dall'offset 160 del file della stanza.

Struttura dei 9 Byte di Definizione per ogni Tile:

Ciascuno dei 6 tipi di Tile (Background, Floor, Wall, Nasty, Ramp, Conveyor) è definito da **9 byte consecutivi**:
- **Byte 0 (Attributo Colore ZX Spectrum)**: Definisce i colori dello schermo nel formato hardware standard: FLASH(1 bit) | BRIGHT(1 bit) | PAPER(3 bit) | INK(3 bit)
- **Byte 1..8 (Bitmap Grafica 8x8 Pixel)**: 8 byte di scanline che compongono l'immagine monocromatica 8×8 del tile.

Etichetta ASM	Offset Stanza	Byte Attributo	Colore Risultante	Byte Bitmap (1..8)
`BallRoomEastBackground`	160	`$07 (%00000111)`	INK 7 (Bianco), PAPER 0 (Nero)	0,0,0,0,0,0,0,0 (Vuoto)
`BallRoomEastFloor`	169	`6 (%00000110)`	INK 6 (Giallo), PAPER 0 (Nero)	170,85,34,68,34,68,34,68
`BallRoomEastWall`	178	`15 (%00001111)`	BRIGHT 1, INK 7 (Bianco Brillante)	68,68,255,17,17,255,85,170
`BallRoomEastNasty`	187	`255 (%11111111)`	FLASH 1, BRIGHT 1, INK 7, PAPER 7	0,0,0,0,0,0,0,0
`BallRoomEastRamp`	196	255	Non usata in BallRoomEast	0,0,0,0,0,0,0,0
`BallRoomEastConveyor`	205	255	Non usata in BallRoomEast	0,0,0,0,0,0,0,0

Il Processo di Mapping e Rendering:
Quando la routine Gfx_DrawRoomLayout disegna la cella alla coordinata (Row, Col):
1. Calcola l'indirizzo nel layout: HL = room_ptr + Row * 8 + (Col / 4).
2. Estrae la coppia di bit per la colonna $Col \pmod 4$. Il risultato è un tipo cella $T \in \{0, 1, 2, 3\}$.
3. Mappa T sul puntatore della Tile corrispondente nella stanza:
   - Se $T = 0 \Rightarrow$ Puntatore a OFFSET_BG (160)

- Se $T = 1 \Rightarrow$ Puntatore a OFFSET_FLOOR (169)
- Se $T = 2 \Rightarrow$ Puntatore a OFFSET_WALL (178)
- Se $T = 3 \Rightarrow$ Puntatore a OFFSET_NASTY (187)

4. Scrive l'**Attributo Colore** nella memoria video degli attributi (area VRAM $5800 + $Row \times 32 + Col$).
5. Scrive gli 8 byte di **Bitmap Pixels** nell'area VRAM dei pixel (area VRAM $4000).

4. Elementi Dinamici Sovrapposti: Rampe e Conveyors

Poiché la griglia di layout a 2 bit supporta soltanto 4 tipi di celle base, **Rampe** e **Nastri Trasportatori (Conveyors)** vengono definiti separatamente come overlay dinamici tramite le strutture ConveyorData e RampData.

Struttura Dati Rampa / Conveyor (4 Byte ciascuna):

- **Byte 0 (Direzione)**: Per rampa: 0 = sale a sinistra, 1 = sale a destra. Per conveyor: 0 = va a sinistra, 1 = va a destra.
- **Byte 1..2 (Location VRAM)**: Indirizzo di memoria buffer attributi $5E00 + offset (es. 24536). Se 0, l'elemento non è presente.
- **Byte 3 (Lunghezza)**: Lunghezza dell'elemento espressa in numero di celle (tiles).

Durante la fase di disegno, la routine Gfx_DrawConveyorsAndRamps legge queste strutture e sovrascrive sia la grafica bitmap sia gli attributi colore direttamente a schermo per tutta la lunghezza specificata.

5. Flusso di Caricamento e Inizializzazione Stanza (LoadRoomAndRestart)

L'intero ciclo di vita del caricamento di una stanza avviene in una sequenza rigorosa orchestrata dal motore principale:

Passo	Routine / Funzione	Operazione Eseguita
1	`PerformRoomChange`	Aggiorna room_ptr con l'indirizzo della nuova stanza (es. BallRoomEastLayout). Imposta la guardia per l'ingresso orizzontale.
2	`UnpackRoomLayout`	Decodifica i 128 byte di layout a 2-bit nel buffer RAM un-packed di 512 byte (unpacked_layout), assegnando a ciascuna cella l'ID 0, 1, 2 o 3 per interrogazioni fisiche rapide.
3	`Gfx_DrawRoomLayout`	Scorre le 512 celle, recupera attributi e bitmap dalle definizioni delle Tile e disegna lo sfondo completo della stanza in VRAM.
4	`Gfx_DrawConveyorsAndRamps`	Disegna gli overlay dinamici di rampe e conveyors a partire dai dati in RampData e ConveyorData.
5	`Items_CheckCollision / Gfx`	Disegna gli oggetti raccoglibili (items) e imposta la mappa delle collisioni degli item.
6	`Enemy_Update / Init`	Inizializza i 8 guardiani/nemici della stanza con i rispettivi colori, limiti di pattugliamento e sprite.
7	`Phys_LoadRoomSettings`	Legge il byte delle impostazioni fisiche (offset 239) per configurare la gravità e la forza del salto per la stanza.
8	`Gfx_PrintPlayer`	Posiziona Willy alle coordinate iniziali (PosWilly) e avvia il loop principale di gioco (Main_loop).